CDN Sentinel
Documentação da Entrega II

Projeto final hands-on • Período 2026.2 • Versão de referência: branch main em 06/11/2026

Integrante	Matrícula
Lael Gustavo Batista Ribeiro de Lima	20230021715
Luis Gustavo Lacerda Reis	20230089650
Josué Guedes Ferreira	20230012313

1 Proposta e objetivo do projeto

O CDN Sentinel é um protótipo acadêmico de rede de distribuição de vídeos que simula clientes em diferentes regiões, servidores de borda e o vazamento de cópias de conteúdo. A cada trecho entregue, a aplicação associa uma prova forense ao cliente e ao servidor, permitindo verificar a origem de uma cópia divulgada indevidamente e propagar uma restrição de acesso entre os nós da CDN.

O objetivo é tornar observáveis aspectos de sistemas distribuídos: processos independentes, comunicação HTTP, descoberta de nós, roteamento por proximidade, failover, propagação de mensagens, autenticação de evidências, injeção de falhas e métricas de capacidade. O console permite iniciar, observar e encerrar a simulação, explorar um mapa da topologia, investigar eventos de segurança e experimentar watermarking sobre um vídeo local.

O ambiente representa um laboratório controlado: cidades, tráfego virtual, custos e atrasos artificiais são modelos didáticos. Não se trata de uma CDN pública, de uma infraestrutura multirregional real nem de um mecanismo antipirataria homologado para produção.

2 Glossário essencial

Termo	Significado neste projeto
CDN e edge node	Rede de distribuição de conteúdo e processo servidor que atende trechos de vídeo perto de um cliente.
Control plane	Backend que cria a topologia, coordena a simulação e oferece a API ao console.
Watermark forense	Identificador associado ao conteúdo para atribuição de origem e verificação de integridade.
HMAC SHA256	Código de autenticação baseado em chave secreta; assina o claim, mas não cifra vídeo.
Gossip e ACK	Propagação de alerta entre pares e confirmação de entrega, com deduplicação e retry.
Failover e SLO	Troca de nó em caso de indisponibilidade e metas configuradas de latência/sucesso.
ADR e C4	Registro de decisão arquitetural e modelo de diagramas de contexto e containers.

3 Diagramas C4

Os diagramas descrevem a topologia lógica. O primeiro identifica o operador e os limites externos; o segundo separa interface, orquestração e processos que entregam conteúdo. No Docker Compose há dois containers de implantação, backend e frontend. Os nós CDN e clientes são processos iniciados pelo backend, e o repositório em memória não é um serviço de banco de dados independente.

3.1 Nível 1 Contexto

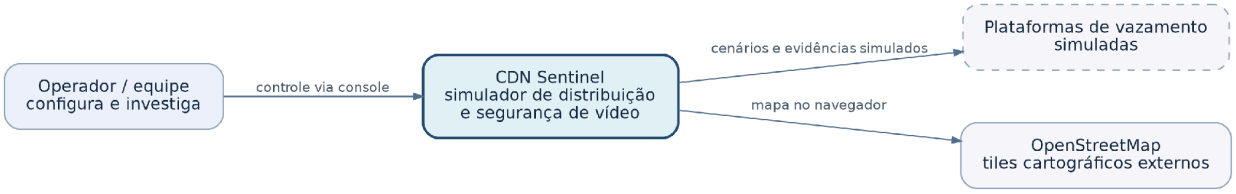


Figura 1 Contexto do CDN Sentinel e integrações externas

3.2 Nível 2 Containers e processos

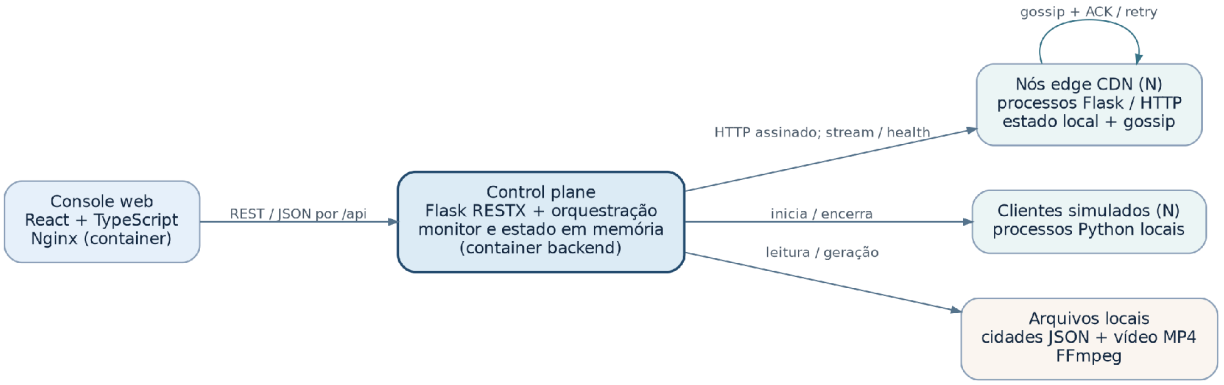


Figura 2 Containers de implantação e processos distribuídos do simulador

Fluxo central: console → API Flask → CDN escolhida por proximidade → chunk autenticado. Uma cópia divulgada na plataforma simulada é encontrada pelo monitor, validada pelo HMAC e associada à blacklist; os nós propagam o alerta por mensagens HTTP com ACK e tentativas posteriores.

4 Registros de decisões arquiteturais

4.1 ADR 0001 Processos HTTP independentes

Status: aceito | Data: 05/11/2026 | Decisores: equipe do projeto

Contexto. Um único processo com chamadas diretas entre classes não evidenciaria falhas de rede, latência nem o estado local dos servidores de borda. A atividade pede uma solução hands-on de comunicação distribuída.

Decisão. O backend Flask mantém o plano de controle e inicializa cada nó CDN como processo Python separado, com porta HTTP própria e estado local. O CDNHttpClient faz chamadas assinadas para health, configuração e stream; os nós trocam alertas de blacklist e telemetria por gossip. Clientes simulados também podem executar em processos locais independentes.

Consequências. Ficam observáveis portas, PIDs, falhas, mensagens e confirmações. Em contrapartida, o gerenciamento de processos e portas é local, a descoberta usa o catálogo da

topologia e o estado desaparece após reinício. O controlador encerra os processos que criou; um orquestrador e service registry reais são evoluções possíveis.

4.2 ADR 0002 Watermark com HMAC SHA256

Status: aceito | Data: 05/11/2026 | Decisores: equipe do projeto

Contexto. Ao localizar uma cópia suspeita, o monitor precisa reconhecer cliente, servidor, instante e trecho, além de rejeitar adulterações na prova sem expor uma chave secreta ao navegador.

Decisão. Cada trecho recebe um token CDNWM1.<payload-base64url>.<assinatura-hex>, com client_id, server_id, issued_at e chunk_id protegidos por HMAC-SHA256. Na demonstração, o token é prefixado ao payload de transporte. Separadamente, o Watermark Studio usa 64 bits derivados do digest HMAC para modificar a luminância de regiões dos frames de um MP4 real com FFmpeg; compara os bits recuperados após blur, ruído e quantização.

Consequências. A prova criptográfica detecta alterações e apoia a atribuição do vazamento. HMAC não criptografa conteúdo, e o canal visual experimental não garante resistência a transformações arbitrárias ou recodificação de produção. A chave é configurável no backend; gestão em cofre, rotação e proteção de mídia em escala ficam para evolução futura.

4.3 ADR 0003 Docker Compose e integração contínua

Status: aceito | Data: 05/11/2026 | Decisores: equipe do projeto

Contexto. A Entrega II exige configuração repetível, frontend integrado ao backend e verificações automáticas que reduzam erros de ambiente.

Decisão. O Compose constrói duas imagens: uma baseada em Python 3.12 com Flask e FFmpeg e outra de build Node 20 que serve o React compilado por Nginx. O Nginx encaminha /api ao serviço backend. O health check do Flask condiciona o início do frontend. O GitHub Actions valida Python, frontend e sintaxe do Compose em push para main e pull requests.

Consequências. Um único comando sobe o laboratório sem dependência de API externa ou download de mídia em runtime; o exemplo Big Buck Bunny já está versionado. O custo é manter Docker e portas locais livres. Não há deploy gradual, registry gerenciado, persistência durável ou assinatura de imagens neste estágio.

5 Tecnologias utilizadas

Camada	Tecnologia	Justificativa e uso atual
API e orquestração	Python 3.12, Flask 3.1, Flask RESTX	REST com Swagger em /docs; inicialização, ciclo de vida e composição de métricas.
Rede interna	HTTP/1.1, JSON, requests	Health checks, streaming, comunicação entre processos e gossip autenticado.
Interface	React 18, TypeScript, Vite 6	Console com páginas de operação, histórico, laboratório e segurança.
Visualização	Leaflet, React Leaflet, Recharts	Mapa dos nós e gráficos temporais de blacklist, tráfego, latência e saúde.

Camada	Tecnologia	Justificativa e uso atual
Segurança	HMAC SHA256, nonce, timestamp, TTL	Prova forense e autenticação de chamadas internas; token opcional de controle.
Mídia e dados	MP4 local, FFmpeg, JSON, memória	Big Buck Bunny, variantes visuais e dados de simulação sem banco externo.
Infraestrutura	Docker, Compose, Nginx, Make	Build e execução padronizados; reverse proxy entre frontend e API.
Qualidade	unittest, compileall, npm build, Actions	Testes backend, checagem de TypeScript/Vite e validação de Compose.

6 Padrões arquiteturais aplicados

Padrão	Aplicação concreta	Evidência no código
API Gateway / fachada BFF	O console acessa uma única API REST, que coordena operações e evita chamadas diretas do navegador aos nós.	app.py, routes/api.py, frontend/src/api.ts
Service Discovery e roteamento	Catálogo de nós com estado de saúde e escolha por proximidade Haversine, com candidatos alternativos.	services/topology_service.py, utils/geo.py
Messaging Fanout e gossip	BLACKLIST_ALERT circula entre pares com message_id, hops, deduplicação, ACK e fila de retry.	node_app.py, services/blacklist_service.py
Aggregator / API Composition	Overview reúne estado, clientes, tráfego, blacklist, leaks, eventos e séries de métricas.	controllers/api_controller.py
Containerized deployment	Imagens independentes de backend e frontend, proxy Nginx e readiness com health check.	docker-compose.yml, Dockerfiles
Ports and adapters	Clientes HTTP, catálogo de conteúdo e repositório isolam transporte, mídia e armazenamento.	services/cdn_http_client.py, services/content_provider.py
Idempotência e failover	Blacklist por cliente e eventos deduplicados; erro ou rejeição local tenta outro nó elegível.	node_app.py, services/stream_service.py

As áreas técnicas cobertas são segurança (prova HMAC, integridade e mensagens assinadas), escalabilidade (nós adicionais, roteamento e capacidade estimada), confiabilidade (failover, retry e idempotência), desempenho (latências e filas) e deployment (Compose e CI). O gateway é uma fachada lógica dentro do Flask, não um produto separado de API gateway.

7 Setup inicial do ambiente

7.1 Requisitos para execução recomendada

- Git e Docker Desktop, ou Docker Engine com o plugin Docker Compose v2. No Windows, executar pelo PowerShell/terminal com Docker Desktop ativo.
- Portas TCP 8080 (console) e 5000 (API) disponíveis. Recursos locais suficientes para construir as imagens e iniciar processos de nós e clientes.
- Não é preciso instalar Python, Node, Nginx ou FFmpeg na máquina quando se utiliza Docker; essas ferramentas entram nas imagens.

7.2 Inicialização com Docker Compose

```
git clone https://github.com/LuisReis09/CDN-Sentinel.git
cd CDN-Sentinel
docker compose config --quiet
docker compose up --build
```

Com os serviços ativos, acessar o console em http://127.0.0.1:8080, a API em http://127.0.0.1:5000, a documentação Swagger em http://127.0.0.1:5000/docs e o health check em http://127.0.0.1:5000/health. No console, iniciar a simulação; os nós e clientes serão criados conforme a configuração escolhida. Para encerrar o ambiente, usar Ctrl+C e executar docker compose down.

Configurações opcionais do Compose: BACKEND_PORT e FRONTEND_PORT alteram as portas publicadas; CDN_SECRET_KEY define a chave HMAC. O valor padrão é apenas para laboratório. A aplicação também admite CDN_CONTROL_TOKEN quando iniciado em um ambiente que repasse essa variável, mas o Compose atual não a encaminha ao backend: o padrão do Compose é modo demo local, sem token de controle.

7.3 Execução de desenvolvimento sem Docker

Requer Python 3.12, Node.js 20 e npm. Em Linux, com o comando make disponível:

```
make install
make dev
```

Para execução manual, instalar requirements.txt, executar python app.py; em outro terminal, executar npm ci e npm run dev dentro de frontend. Nesse modo, o Vite fica em http://127.0.0.1:5173 e encaminha /api ao backend. O Makefile também oferece make test, make build e make docker-down.

7.4 Verificação do setup e CI

A equipe pode rodar make docker-smoke em Linux: o alvo constrói e sobe os containers, aguarda /health, testa a rota de demonstração de watermark e o frontend pelo Nginx, e executa docker compose down ao final. Para checar o código sem containers, utilizar make test. O workflow do GitHub Actions executa três jobs: backend (compileall e unittest), frontend (npm ci e build) e compose-config (docker compose config --quiet).

Evidência verificada na main: o workflow CI da revisão c90c28d terminou com sucesso nos três jobs em 06/11/2026 no horário de Brasília (07/11 em UTC). Esse resultado não equivale a um ensaio de carga em produção nem garante que o smoke Docker tenha rodado no CI.

8 Divisão de responsabilidades

Integrante	Atuação registrada na equipe
Luis Gustavo Reis de Lacerda	Desenvolvimento do código e decisões sobre arquitetura e stack tecnológico.
Josué Guedes Ferreira	Documentação técnica e apoio às decisões de ADRs e padrões arquiteturais.
Lael Gustavo Batista Ribeiro de Lima	Apoio à documentação, controle de qualidade e desenho inicial da arquitetura.

As responsabilidades acima correspondem aos três integrantes identificados no material da equipe. A orientação geral da disciplina prevê grupos de cinco pessoas; a composição oficial deve ser confirmada com o professor, sem atribuir nomes ou atividades inexistentes.

9 Plano de validação e demonstração

Cenário	Procedimento e evidência esperada
Fluxo de streaming	Iniciar 2 a 3 nós e clientes; solicitar chunk; observar servidor escolhido, request_id, trace_id e tempos artificial, HTTP e ponta a ponta.
Falha e resiliência	Marcar nó como offline no Network Lab; repetir pedido; observar outro candidato e o motivo do failover. Restaurar o nó ao final.
Detecção e gossip	Simular vazamento; executar/aguardar scan; conferir prova, blacklist, mensagens de propagação, ACKs e eventuais retries.
Integridade forense	Gerar token no Watermark Studio e verificar claim; adulterar assinatura; verificar rejeição. Comparar original, marcado e diferença visual.
Capacidade e observabilidade	Ajustar nós, tier, demanda e preço; acompanhar capacidade estimada, pressão, fila, custos, p50/p95/p99 e gráficos históricos.
Automação	Rodar unittest, build frontend, validação do Compose e smoke local; anexar evidências se exigidas na Entrega III.

O modelo de capacidade usa perfis basic, standard, pro e enterprise e considera gargalos estimados de CPU e banda; a fila e rejeições são acompanhadas por nó. Os valores financeiros em USD, o preço por cliente e o SLO são parâmetros de um cenário fictício, não medições de infraestrutura comercial. O histórico temporal mantém amostras em memória, com retenção limitada.

10 Trade offs e evolução prevista

- Memória em vez de banco persistente: permite inspeção rápida e menos dependências, mas perde histórico ao reiniciar e não suporta operação distribuída durável.
- HTTP e processos locais em vez de mensageria ou cluster remoto: expõem as fronteiras entre serviços e o protocolo, mas ainda dependem de uma só máquina de execução.
- HMAC e marcação visual experimental: provam integridade do claim e permitem um estudo reproduzível; não equivalem a criptografia do vídeo ou detector forense industrial.
- Modelo de capacidade e custos: oferece diagnóstico para decisões de escalonamento; estimativas de demanda, filas e receitas precisam ser validadas sob carga real.

Para a Entrega III, as evoluções mais relevantes são testes de carga e resiliência com resultados quantitativos, persistência durável, controles de produção para a chave e identidade, e evidências de comportamento sob falhas. O escopo atual já cobre os artefatos iniciais exigidos: C4, três ADRs, justificativa de stack, padrões, setup com Compose, CI e responsabilidades.

11 Ferramentas de IA e fontes de verificação

O desenvolvimento do CDN-Sentinel é conduzido integralmente pelos integrantes da equipe, responsáveis pela concepção das ideias, definição dos requisitos, planejamento das funcionalidades, decisões arquiteturais, implementação e validação do projeto. O ChatGPT/Codex é utilizado como ferramenta de apoio técnico, auxiliando na análise de requisitos, elaboração e revisão da documentação, sugestões de implementação, configuração do ambiente Docker/CI e estruturação de testes.

Todas as atividades realizadas com auxílio de IA são orientadas, acompanhadas e supervisionadas pela equipe. As decisões sobre o que desenvolver, como implementar e quais soluções adotar permanecem sob responsabilidade humana. Nenhuma sugestão ou alteração é incorporada sem avaliação prévia: o código, as configurações, os documentos e os resultados são continuamente inspecionados, testados e corrigidos pelos integrantes. Esse processo de vistoria ocorre durante todo o ciclo de desenvolvimento, desde a concepção das funcionalidades até sua integração ao projeto.

Dessa forma, a IA atua exclusivamente como ferramenta de suporte e produtividade, sem substituir o raciocínio crítico, a autoria intelectual ou a responsabilidade técnica da equipe. O uso das ferramentas, as orientações fornecidas e a avaliação dos resultados estão documentados em docs/ai-tools.md.

12 Base de verificação e rastreabilidade

A documentação e as informações técnicas foram confrontadas com a implementação disponível no repositório LuisReis09/CDN-Sentinel, considerando a branch main, na revisão c90c28dd602ef6315a42e9fdd74bf97b8fa8f114.

A conferência abrangeu o código-fonte, os diagramas de arquitetura, os registros de decisões arquiteturais (ADRs), a definição do stack tecnológico, o plano de testes e os arquivos de infraestrutura e integração contínua. Também foi considerado o resultado da execução nº 42320819968 do GitHub Actions, concluída com sucesso.

A elaboração do documento seguiu os requisitos estabelecidos pelo professor no enunciado do Projeto Final de Engenharia de Sistemas Distribuídos 2026.2, com foco nas especificações da Entrega II.

Repositório: *https://github.com/LuisReis09/CDN-Sentinel*

CI: *https://github.com/LuisReis09/CDN-Sentinel/actions/runs/42320819968*